

Test Results — PropertyGenie

Document version: 1.0 **Test date:** 2025-06-06 **Environment:** Node.js 20 · Next.js 12 · React 17 · Material UI 5 · Ubuntu 22.04 **Run command:** `npm run dev` (manual QA) + component render checks

1. Overview

PropertyGenie is a responsive property-listing web application built with Next.js and Material UI. This document records the manual QA pass and component-level checks performed against the listing flow: hero search, the multi-level category filter, the property grid, pagination, and responsive layout.

The catalog layer runs against a 50-item mock dataset (`components/mockData.js`) that mirrors the shape of the live REST payload — each property carries `name` , `image` , `address` , `room.bed` , `room.bath` , and a Malaysian-Ringgit-formatted `rent.price` . This lets the UI be exercised end-to-end without a live backend.

2. Component Render Checks

Component	What was verified	Result
<code>HeroSection</code>	Renders hero image, headline, and search entry point; CTA routes to <code>/ listings</code>	✓ Pass
<code>FilterBar</code>	Category → Type → Subtype hierarchy renders; residential/commercial branches expand correctly	✓ Pass
<code>PropertyCard</code>	Image with fallback handling, name, address, price, bed/bath chips; hover lift transition	✓ Pass
<code>FooterInfo</code>	Company links and contact block render on all breakpoints	✓ Pass
<code>listings.js</code> page	Grid populates from data source, pagination controls present	✓ Pass

3. Filter Logic

The category model is a three-level hierarchy defined in `FilterBar.js` :

```
Residential
├─ Bungalow / Villa
```

```
|   └─ bungalow-house, link-bungalow, zero-lot-bungalow, twin-villas, twin-courtyard-villa
└─ Apartment / Condo / Service Residence
    └─ condominium, apartment, flat, penthouse, service-residence, studio, duplex, townhouse-condo
Commercial
└─ (office, retail, industrial branches)
```

Cases verified:

Case	Input	Expected	Result
Category select	Choose "Residential"	Type dropdown populates with residential types only	✓ Pass
Type → subtype cascade	Residential → Apartment/Condo	Subtype checkboxes appear for that type	✓ Pass
Router sync	Apply filter	Selection reflected in URL query via <code>next/router</code>	✓ Pass
Reset	Clear filters	Grid returns to full 50-item set	✓ Pass

4. Pagination & Grid

Case	Input	Expected	Result
Page size	Default load	Grid shows first page of results	✓ Pass
Next / Prev	Navigate pages	Correct slice of the 50-item set renders	✓ Pass
Boundary	Last page	Next control disabled, no empty page	✓ Pass
Image fallback	Property with missing image	Falls back to <code>/fallback-property.jpg</code> without layout break	✓ Pass

5. Responsive Layout

Card width is defined responsively as `{ xs: '100%', sm: '85%', md: '90%' }`. Breakpoints were checked in the browser device toolbar.

Breakpoint	Width	Result
Mobile	375px	Single-column stack, filters collapse cleanly
Tablet	768px	Two-column grid, hero scales
Desktop	1280px	Multi-column grid, full filter bar

6. Summary

Area	Cases	Pass	Fail
Component render	5	5	0
Filter logic	4	4	0
Pagination & grid	4	4	0
Responsive	3	3	0
Total	16	16	0

All checks passed. The listing flow behaves correctly against the mock dataset, the category filter cascades through all three levels, pagination respects boundaries, and the layout holds from 375px to desktop. The data layer is drop-in compatible with a live REST endpoint via the existing `axios` client.